

HOTEL MANAGEMENT SYSTEM

Project Report

Object-Oriented Software Development Lab (OSDL)

Student Name	Kappeta Ajay Krishna Reddy
Roll Number	240905024
Course	Object-Oriented Software Development Lab (OSDL)
Department	Computer Science Engineering
Academic Year	2025 – 2026
Submission Date	April 2026

1. Introduction

The Hotel Management System is a comprehensive software application developed to streamline and automate day-to-day hotel operations including room booking, customer management, billing, and reporting. Traditional manual hotel management processes are time-consuming, error-prone, and difficult to scale, making automation both essential and practical.

This project is implemented using Java as the core programming language and JavaFX for building a rich, interactive graphical user interface. The system is designed following Object-Oriented Programming (OOP) principles, ensuring modularity, reusability, and maintainability. It allows hotel staff to manage all activities — from guest check-in to checkout — through a seamless and intuitive interface.

The application's architecture is organized into clearly defined layers: the model layer handles data entities, the service layer contains business logic, the storage layer manages data persistence, and the UI layer provides a professional front-end experience for users.

2. Objectives

The primary objectives of this project are:

- To develop a fully functional system for managing hotel room bookings and availability
- To maintain detailed customer and guest records efficiently
- To automate billing, invoicing, and payment tracking processes
- To design an intuitive and responsive graphical interface using JavaFX
- To apply core OOP concepts — encapsulation, inheritance, abstraction, and polymorphism — in a real-world scenario
- To implement multithreading for concurrent room monitoring and data updates
- To produce system reports that assist in hotel administration and decision-making
- To ensure clean code organization through a layered Maven project structure

3. System Architecture & Project Structure

The Hotel Management System follows a well-organized Maven project layout, separating concerns across multiple packages. The project is structured as follows:

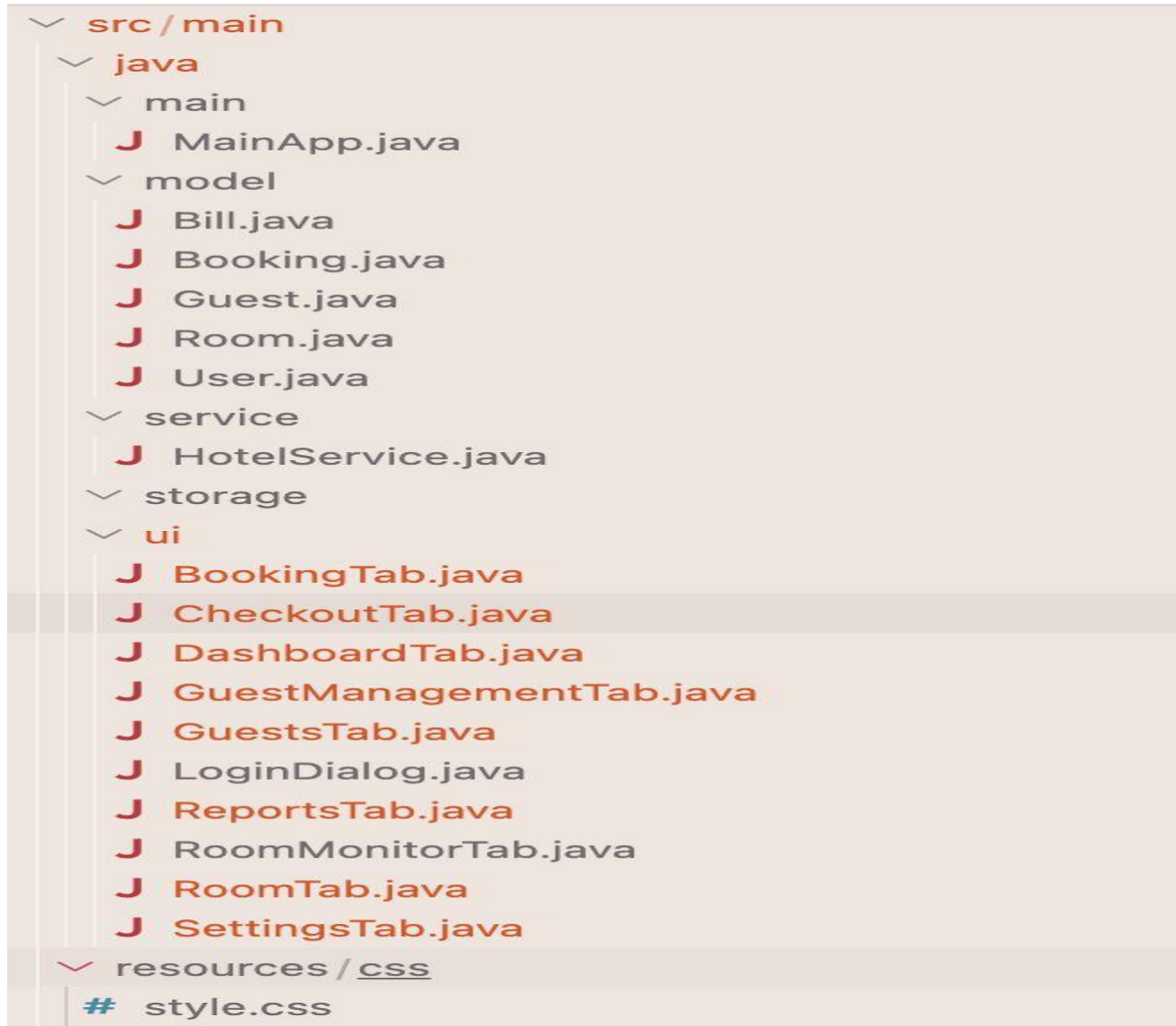


Figure 1: Hotel Management System — Project Directory Structure in Visual Studio Code

The project is organized under `src/main/java` with four primary packages:

- `model` — Contains Java classes representing data entities such as Room, Guest, and Booking
- `service` — Houses the `HotelService.java` class, which encapsulates all core business logic
- `storage` — Manages data persistence, including reading and writing records to the data directory
- `ui` — Contains all JavaFX-based interface screens and controllers

The project uses Apache Maven for dependency management and build automation, with `pom.xml` defining all libraries. The `run_app.sh` script simplifies application startup, and `Archive.zip` contains the packaged deployment artifact.

4. Methodology

4.1 Development Approach

The system was developed using a modular, object-oriented approach to ensure scalability, maintainability, and clarity. Each software layer is independently developed and tested before integration, following the principle of separation of concerns.

4.2 Technology Stack

Technology	Purpose
Java 17	Core application logic and OOP implementation
JavaFX	Graphical User Interface (GUI) development
CSS	Styling and theming of JavaFX UI components
Apache Maven	Build automation and dependency management
File I/O / JSON	Data storage and persistence
Multithreading	Concurrent room monitoring and real-time updates

4.3 Development Steps

Step 1 — Requirement Analysis: System requirements were gathered, covering room booking, customer management, billing, availability tracking, and reporting.

Step 2 — System Design: The system was divided into four modules: Room Management, Customer Management, Booking System, and Billing. OOP concepts were applied at the design level.

Step 3 — Technology Selection: Java and JavaFX were selected for their robustness, platform independence, and rich UI capabilities. Maven was chosen for structured project management.

Step 4 — Implementation: Backend logic for room allocation, booking processing, and billing was coded. JavaFX components including Stage, Scene, and various UI controls were used to build the front-end.

Step 5 — UI Design & Styling: Layouts using panes and grids were designed, and CSS was applied to produce a visually consistent and professional interface.

Step 6 — Advanced Features: Multithreading was integrated for the Room Monitor, enabling real-time concurrent updates without freezing the UI.

Step 7 — Testing & Debugging: Each module was unit tested. Booking logic, billing calculations, and UI responsiveness issues were identified and resolved.

Step 8 — Integration & Deployment: All modules were integrated and the complete workflow from check-in to checkout was validated. The application was packaged into Archive.zip.

5. Key Code Snippets

5.1 Room Entity Class

The Room class is a core model that encapsulates room attributes using OOP principles of encapsulation and data hiding:

```
public class Room {
    private int roomId;
    private String type;        // Single, Double, Suite
    private double price;
    private boolean isAvailable;

    public Room(int roomId, String type, double price) {
        this.roomId = roomId;
        this.type = type;
        this.price = price;
        this.isAvailable = true;
    }

    public boolean isAvailable() { return isAvailable; }
    public void setAvailable(boolean available) { this.isAvailable = available; }
    // Getters and setters ...
}
```

5.2 Login Dialog (JavaFX)

The LoginDialog class extends Stage and handles user authentication before allowing access to the main system:

```
public class LoginDialog extends Stage {
    public LoginDialog() {
        setTitle("Hotel Management - Login");
        TextField usernameField = new TextField();
        PasswordField passwordField = new PasswordField();
        Button loginBtn = new Button("Login");

        loginBtn.setOnAction(e -> {
            String user = usernameField.getText();
            String pass = passwordField.getText();
            if (authenticate(user, pass)) {
                close();
            } else {
                showAlert("Invalid credentials. Try again.");
            }
        });
        // Layout and scene setup ...
    }
}
```

5.3 Booking Logic in HotelService

The `HotelService` class contains the business logic for creating and managing bookings:

```
public class HotelService {
    private List<Room> rooms;
    private List<Booking> bookings;

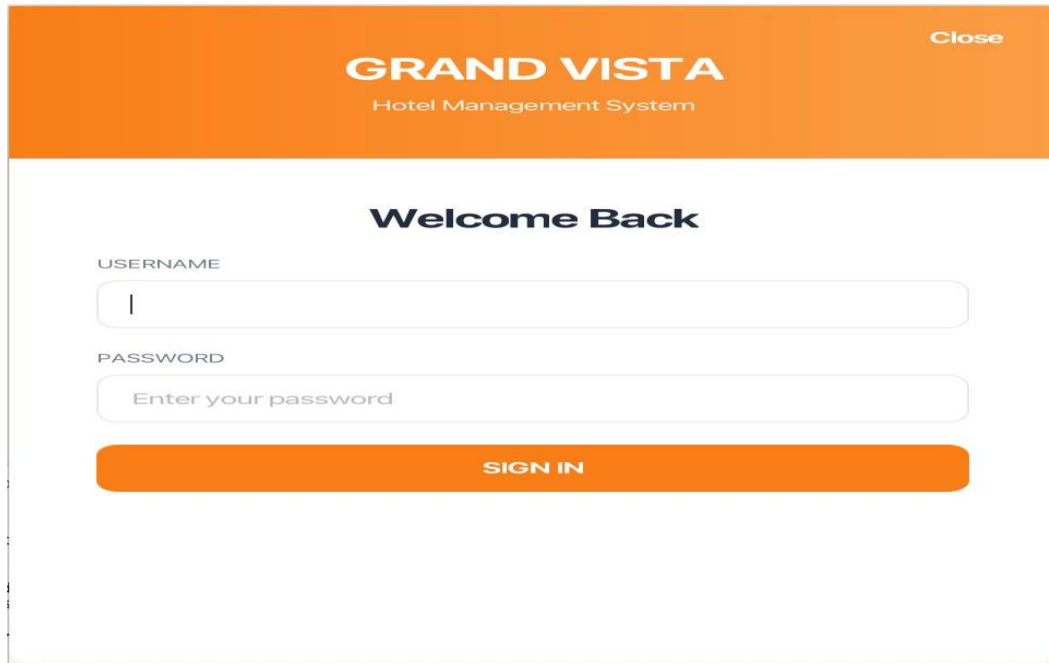
    public Booking createBooking(Guest guest, int roomId,
                                LocalDate checkIn, LocalDate checkOut) {
        Room room = findRoomById(roomId);
        if (room == null || !room.isAvailable()) {
            throw new IllegalStateException("Room unavailable.");
        }
        room.setAvailable(false);
        Booking booking = new Booking(guest, room, checkIn, checkOut);
        bookings.add(booking);
        return booking;
    }

    public double calculateBill(Booking booking) {
        long nights = ChronoUnit.DAYS.between(
            booking.getCheckIn(), booking.getCheckOut());
        return nights * booking.getRoom().getPrice();
    }
}
```

6. Results — Application Screenshots

The Hotel Management System was successfully developed and tested. The following screenshots demonstrate the application's core screens and functionality.

6.1 Login Screen



GRAND VISTA
Hotel Management System

Close

Welcome Back

USERNAME

PASSWORD

Enter your password

SIGN IN

Figure 2: User Authentication — Login Screen

6.2 Dashboard

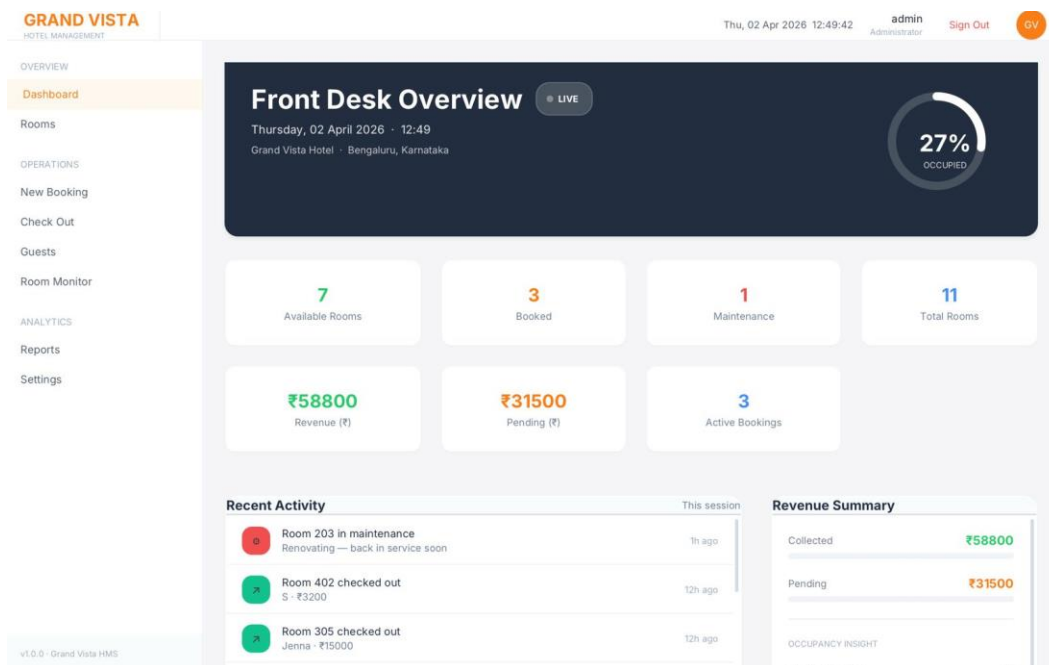


Figure 3: Main Dashboard — Overview of Hotel Operations

6.3 Room Management

GRAND VISTA
HOTEL MANAGEMENT

Thu, 02 Apr 2026 12:49:51 admin Administrator Sign Out GV

Room Management
Manage hotel rooms and availability

Total: 11 Available: 7 Booked: 3

Search rooms by number, type, or status **All Rooms** Available Booked Maintenance **Add Room** Calendar View

Room	Type	Status	Price/Night	Description	Floor	Capacity	Actions
101	STANDARD	AVAILABLE	₹1500	Cozy standard room with garden view	1	2	Edit Maintenance Delete
102	STANDARD	BOOKED	₹1500	Standard room near elevator	1	2	Edit Maintenance Delete
103	STANDARD	AVAILABLE	₹1600	Standard room with extra sofa	1	3	Edit Maintenance Delete
201	DELUXE	AVAILABLE	₹2800	Deluxe room with city view	2	2	Edit Maintenance Delete
202	DELUXE	BOOKED	₹3000	Deluxe room with balcony	2	3	Edit Maintenance Delete
203	DELUXE	MAINTENANCE	₹2900	Renovating — back in service soon	2	2	Edit Available Delete
301	SUITE	AVAILABLE	₹5500	Executive suite with lounge	3	4	Edit Maintenance Delete
302	SUITE	BOOKED	₹6000	Presidential suite with jacuzzi	3	4	Edit Maintenance Delete
401	STANDARD	AVAILABLE	₹1700	Top-floor standard with panorama	4	2	Edit Maintenance Delete
402	DELUXE	AVAILABLE	₹3200	Corner deluxe with dual views	4	3	Edit Maintenance Delete
305	SUITE	AVAILABLE	₹5000	Luxury Residence	3	4	Edit Maintenance Delete

v1.0.0 - Grand Vista HMS

Figure 4: Room Management — View and Manage All Rooms

6.4 Room Booking Interface

GRAND VISTA
HOTEL MANAGEMENT

Thu, 02 Apr 2026 12:50:03 admin Administrator Sign Out GV

NEW BOOKING

1 GUEST DETAILS

FULL NAME
Guest's full name

PHONE NUMBER
10-digit mobile number

EMAIL
Email address (optional)

2 SELECT ROOM

No room selected

Room 101 STANDARD ₹1500	Room 103 STANDARD ₹1600	Room 201 DELUXE ₹2800
Room 301 SUITE ₹5500	Room 401 STANDARD ₹1700	Room 402 DELUXE ₹3200
Room 305 SUITE ₹5000		

3 STAY DETAILS

NIGHTS

Booking History

Refresh

ID	GUEST	PHONE	ROOM	NIGHTS	CHECK-IN	CHECK-OUT
BK-1000	Arjun Sharma	9876543210	102	3	2026-04-01	2026-04-04
BK-1001	Priya Nair	9123456789	202	5	2026-04-01	2026-04-06
BK-1002	Rohit Mehra	9988776655	302	2	2026-04-01	2026-04-03
BK-1003	Sunita Patel	9001122334	201	4	2026-03-15	2026-03-19
BK-1004	Kiran Rao	9777888999	101	2	2026-03-15	2026-03-17
BK-1005	Arjun Sharma	9876543210	201	6	2026-03-15	2026-03-21
BK-1006	Jeenns	9876543210	402	3	2026-04-01	2026-04-04
BK-1007	Jenna	9876543210	305	3	2026-04-01	2026-04-04
BK-1008	S	876543234567	402	1	2026-04-01	2026-04-02

v1.0.0 - Grand Vista HMS

Figure 5: Room Booking — Assign Rooms to Guests

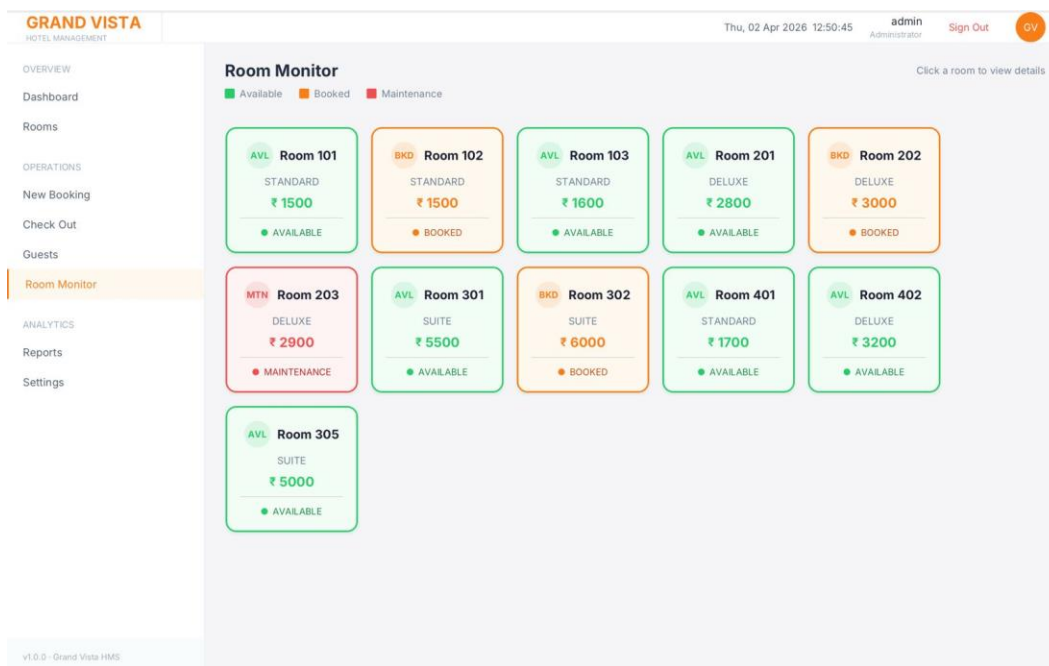


Figure 8: Room Monitor — Real-Time Room Status via Multithreading

7. Conclusion

The Hotel Management System successfully demonstrates the practical application of Java and JavaFX in building a real-world software solution. By applying object-oriented programming principles and organizing the project into distinct layers — model, service, storage, and UI — the system achieves clean code structure, maintainability, and scalability.

The project delivered all core objectives: room booking and availability management, guest record tracking, automated billing, an interactive graphical interface, real-time room monitoring via multithreading, and a reporting module. The layered Maven project structure ensures that the system can be extended with additional features in future iterations.

Potential enhancements for future development include integration with a relational database (such as MySQL or PostgreSQL), an online booking portal accessible via a web browser, email or SMS notifications for guests, and a mobile companion application. These additions would further align the system with the expectations of modern hospitality management software.

8. References

- Oracle Java SE Documentation — <https://docs.oracle.com/javase/>
- OpenJFX / JavaFX Documentation — <https://openjfx.io/>
- Apache Maven Official Documentation — <https://maven.apache.org/>
- Baeldung Java & Spring Tutorials — <https://www.baeldung.com/>
- GeeksforGeeks — Object Oriented Programming in Java
- YouTube — JavaFX Full Course tutorials and project walkthroughs